本节内容提要

Apriori算法改进

- Apriori算法核心回顾
- Apriori算法的瓶颈
- 提高Apriori算法的技术
- Close算法

Apriori算法核心

1.算法运行过程

（1）简单统计所有含一个元素项目集出现的频率,并找出那些不小于最小支持度的项目集,即一维最大项目集.

（2）～（k）：根据第（k-1）步生成的k-1为最大项目集产生k维侯选项目集,然后对数据库进行搜索,得到侯选项目集的项集支持度,与最小支持度比较,从而找到k维最大项目集.

Apriori算法例子

- Minsupport=50%

Database D

TID	Items
100	1 3 4
200	2 3 5
300	1 2 3 5
400	2 5

Scan D →

C_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{4}	1
{5}	3

→ L_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{5}	3

C_2

itemset
{1 2}
{1 3}
{1 5}
{2 3}
{2 5}
{3 5}

Scan D ←

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

← L_2

itemset	sup
{1 3}	2
{2 3}	2
{2 5}	3
{3 5}	2

C_3

itemset	sup
{1 2 3}	1
{1 2 3 5}	
{2 3 5}	2
{1 3 5 }	1

Scan D → L_3

itemset	sup
{2 3 5}	2

Apriori算法的性能瓶颈

- Apriori作为经典的频繁项目集生成算法，在数据挖掘中具有里程碑的作用。
- Apriori算法有两个致命的性能瓶颈：
  - 1．多次扫描事务数据库，需要很大的I/O负载
    - 对每次k循环，侯选集C_k中的每个元素都必须通过扫描数据库一次来验证其是否加入L_k。假如有一个频繁大项目集包含10个项的话，那么就至少需要扫描事务数据库10遍。
  - 2．可能产生庞大的侯选集
    - 由L_{k-1}产生k-侯选集C_k是指数增长的，例如10^4个1-频繁项目集就有可能产生接近10^7个元素的2-侯选集。如此大的侯选集对时间和主存空间都是一种挑战。

提高Apriori算法效率的技术

- 一些算法虽然仍然遵循Apriori 属性，但是由于引入了相关技术，在一定程度上改善了Apriori算法适应性和效率。
- 主要的改进方法有：
  - 基于数据分割（Partition）的方法：基本原理是“在一个划分中的支持度小于最小支持度的k-项集不可能是全局频繁的”。
  - 基于散列（Hash）的方法：基本原理是“在一个hash桶内支持度小于最小支持度的k-项集不可能是全局频繁的”。
  - 基于采样（Sampling）的方法：基本原理是“通过采样技术，评估被采样的子集中，并依次来估计k-项集的全局频度”。
  - 其他：如，动态删除没有用的事务：“不包含任何L_k的事务对未来的扫描结果不会产生影响，因而可以删除”。

基于数据分割的方法

- **定理3-5** 设数据集D被分割成分块D_1，D_2，…，D_n，全局最小支持数为minsup_count。如果一个数据分块D_i的局部最小支持数$minsup_count_i$（i=1，2，…，n），按着如下方法生成：

  $minsup_count_i = minsup_count * ||D_i|| / ||D||$

则所有的局部频繁项目集涵盖全局频繁项目集。

- **作用：**
  - 1．**合理利用主存空间：**数据分割将大数据集分成小的块，为块内数据一次性导入主存提供机会。
  - 2．**支持并行挖掘算法：**每个分块的局部频繁项目集是独立生成的，因此提供了开发并行数据挖掘算法的良好机制。

基于散列的方法

- 1995，Park等发现寻找频繁项目集的主要计算是在生成2-频繁项目集上。因此，Park等利用了这个性质引入杂凑技术来改进产生2-频繁项目集的方法。
- 例子： 桶地址 = （10x+y）mod 7；minsupport_count=3

TID	Items
1	I1，I2，I5
2	I2，I4
3	I2，I3
4	I1，I2，I4
5	I1，I3
6	I2，I3
7	I1，I3
8	I1，I2，I3，I5
9	I1，I2，I3

桶地址	0	1	2	3	4	5	6
桶计数	2	2	4	2	2	4	4
桶内	{I1，I4}	{I1，I5}	{I2，I3}	{I2，I4}	{I2，I5}	{I1，I2}	{I1，I3}
	{I3，I5}	{I1，I5}	{I2，I3}	{I2，I4}	{I2，I5}	{I1，I2}	{I1，I3}
			{I2，I3}			{I1，I2}	{I1，I3}
			{I2，I3}			{I1，I2}	{I1，I3}

L2={{I2，I3}， {I1，I2}， {I1，I3}}

探索新的理论

- 随着数据库容量的增大，重复访问数据库（外存）将导致性能低下。因此，探索新的理论和算法来减少数据库的扫描次数和侯选集空间占用，已经成为近年来关联规则挖掘研究的热点之一。
- 两个典型的方法：
  - Close算法
  - FP-tree算法

Apriori算法改进

对Apriori算法改进方向：

探索新的理论和算法来减少数据库的扫描次数和侯选集空间占用，这里主要介绍：

-------Close算法

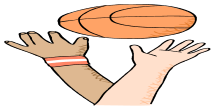
Close算法对应的原理

1.Close算法改进方向：

加速频繁项目集的生成，减少数据库库的扫描次数。

2.Close算法改进基于的基本原理：

一个频繁闭合项目集的所有闭合子集一定是频繁的；一个非频繁闭合项目集的所有闭合超集一定是非频繁的。



3.基本概念

（1）子集和超集：

对于两个集合A与B，如果集合A的任何一个元素都是集合B的元素，而集合B中至少有一个元素不属于集合A，则称集合A是集合B的真子集，集合B为集合A的超集。

（2）频繁项集

若S的支持度≥给定最小支持度，称S为频繁项集(Frequent Itemset)。t为一条事务，如果S⊆t，则称事务t包含S。

（3）最大频繁项集

如果频繁项集L的所有超集都是非频繁项集，那么称L为最大频繁项集

（4）闭项集和频繁闭项集

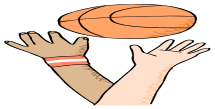
所谓闭项集，就是指一个项集X，它的直接超集的支持度计数都不等于它本身的支持度计数。如果闭项集同时是频繁的，也就是它的支持度大于等于最小支持度阈值，那它就称为频繁闭项集。

例如，有交易数据库：

TID	item
1	abc
2	abcd
3	bce
4	acde
5	de

因为项集{b,c}出现在TID为1,2,3的事务中，所以{b,c}的支持度计数为3。而{b,c}的直接超集：{a,b,c},{a,b,c,d}的支持度计数分别为2,1，都不等于{b,c}的支持度计数3，所以{b,c}为闭项集，如果支持度阈值为40%，则{b,c}也为闭频繁项集。

项集{a,b}出现在TID为1,2的事务中，其支持度计数为2。而它的直接超集{a,b,c}支持度计数也为2，所以{a,b}不是闭项集。



4.close算法具体步骤：

（1）查找频繁闭项集

（2） 生成频繁项目集

（3）生成关联规则

查找频繁闭合项集：

利用频繁闭合i-项集为FC_i，生成闭合（i+1）-项集，也就是 FC_{i+1}。基本过程大致描述为：首先找到候选1-项集，记为:FCC_1，扫描数据库找到它的闭合以及support，得到候选闭合项集，修剪以后，得到FC_1，与自身连接以后，得到FCC_2，继续得到FC_2…继续下去，知道FCC_r为空，算法结束，得到频繁闭合项集。

5.算法实例分析

给出实例数据库，如下图所示：其中minsupport=2;

TID	Itemset
1	A，B，E
2	B，D
3	B，C
4	A，B，D
5	A，C
6	B，C
7	A，C
8	A，B，C，E
9	A，B，C

（1）计算 FCC_1 各个产生式的闭合与支持度。

首先得到 FCC_1 产生式：{A}、{B}、{C}、{D}、{E}。计算闭合集。

1：{ABE} 和{A}的闭合为：{ABE}

4：现有A的闭合{ABE}和{ABD}取交集得到{A}闭合{AB}：

5：现有A的闭合{AB}和{AC}取交集得到{A}的闭合：{A}

7：现有A的闭合{A}和{AC}取交集得到{A}的闭合：{A}

8：现有A的闭合{A}和{ABCE}取交集得到{A}的闭合：{A}

9：现有A的闭合{A}和{ABC}取交集得到{A}的闭合：{A}

同时计算{A}的支持度为6.同理得到{B}、{C}、{D}、{E}的闭合。列表如下：

产生项	闭合集	支持度
{A}	{A}	9
{B}	{B}	7
{C}	{C}	6
{D}	{BD}	2
{E}	{ABE}	2

（2）进行修剪，减掉支持度小于2的，得到FC_1，本例和FCC_1相同。

（3）利用FC_1 的产生项生成FCC_2

生成2-项目集，和apriori算法一样，然后将FC_1中是FCC_2的某个候选项选择出来，记为Sp，如果FCC_1的这一项是Sp字母的闭合项则删除，得到FCC_2 ,计算如下:

Close算法的例子1

2-项目集为：{AB}{AC}{AD}{AE}{BC}{BD}{BE}{CD}{CE}{DE}

利用FC1筛选：{AE}和{BE}是{E}闭合{ABE}的子集，删除。

得到FCC2：{AB,AC,AD,BC,BD,CD,CE,DE}

(4) 计算各产生式的闭合支持度

产生项	闭合集	支持度
{AB}	{AB}	4
{AC}	{AC}	4
{AD}	{ABD}	1
{BC}	{BC}	4
{BD}	{BD}	2
{CE}	{ABCE}	1

Close算法的例子1

(5)修剪得到FC_2={AB, AC, BC, BD}

(6)继续产生FCC_3={ABC}，支持度为2. 修剪以后得到FC_3={ABC}.

(7)生成FCC_4，为空算法结束。得到所有不重复的闭合FC。

FC={A, B, ABE, BD, C, AB, AC, BC, ABC}.

(8)所有闭合集统计：L3={ABC, ABE}, L2={AB, AC, BC, BD}, L1={A, B, C}

。最多的个数为3。

L_3频繁项分解， 先分解{ABC}为{AB, AC, BC}全部在FC中，不用添加

。然后分解{ABE}为{AB, AE, BE}, 将{AE, BE}加入L_2。得到L2为

={BD, AB, AC, BC, AE, BE}

L_2项分解：{A, B, C, D, E}，加入L_1。

得到的L1为={A, B, C, D, E};

最终L3={ABC, ABE};

练习：用Close算法求频繁项集

样本数据库

TID	Itemset
1	A，B，C，D
2	B，C，E
3	A，B，C，E
4	B，D，E
5	A，B，C，D

Close算法的例子2

- Close算法作用到数据集的执行过程（假如 *minsup_count*=3）：

样本数据库

TID	Itemset
1	A，B，C，D
2	B，C，E
3	A，B，C，E
4	B，D，E
5	A，B，C，D

- 扫描数据库得到L_1={(A,3), (B,5), (C,4), (D,3), (E,3)}；相应关闭项目集为 C_l (A)={ABC,3}，C_l (B)={B,5}，C_l (C)={BC,4}，C_l (D)={BD,3}，C_l(E)={BE,3}；
- L_2={(AB,3), (AC,3), (BC,4), (BD,3), (BE,3)}；相应关闭集为C_2 (AB)={ABC,3}；
- L3，L4，L5不用测，于是频繁大项集为{ABC}。

6.Close算法与Apriori算法比较

Apriori算法中，每个元素都需要扫描整个数据库来验证是否加入频繁项集。增加了I/O负载。

close算法采用频繁闭合项目集来发现频繁项集，在一定程度上减少了数据库的扫描次数，减少了查找空间。

R语言实现

- R语言关联规则包：arules.

- 参考教材：
  机器学习与R语言（第二版），第8章.

- 本节课程实例代码：
  http://shfkuang.github.io/dm/apriori/apriori.html

close算法R语言实现

close算法代码是在上一节的apriori算法基础上改进的！

函数示例如下：

对data采用apriori找出所有闭频繁项集，
##target=closed！！！表示用close算法

```
closed <- apriori(data, parameter = list(target = "closed", support = 0.4))
```

Thank you !!!